

Performance Analysis of No-Code Neural Network Text Classification for Customer Intent Routing

Nanelle Tuguinay
College of Computer Studies and
Engineering
Jose Rizal University
Pasig City, Philippines
nanelle.tuguinay@my.jru.edu

Abstract—This report examines the operational performance, edge-case limitations, and underlying processing pipeline of a shallow neural network trained to classify customer feedback into three distinct intent categories: Positive_Praise, Negative_Complaint, and Urgent_Support. Utilizing a no-code supervised learning environment (*Machine Learning for Kids*), a dataset of 30 distinct training examples was established to construct mathematical associations between text patterns and labels. The model was evaluated against a test set of 10 unseen phrases, achieving an overall classification accuracy of 60.00%. Key findings highlight significant vulnerability to sarcastic phrasing and out-of-vocabulary queries, while demonstrating high confidence on clear, semantically direct inputs. Additionally, the feature extraction process within the neural architecture is mapped and analyzed.

Keywords—Deep Learning, Text Classification, Neural Networks, Natural Language Processing, Sentiment Analysis, Feature Extraction.

I. INTRODUCTION

Automated sentiment analysis and intent classification are critical components of modern customer support workflows [1]. Traditional software systems relied heavily on manual, hardcoded rules or keyword matching [1]. However, modern deep learning architectures allow systems to learn mathematical mappings directly from tagged datasets without human-written logic [1], [2].

In this laboratory activity, a supervised text classification model was implemented using the *Machine Learning for Kids* web application [1], [3]. The objective was to evaluate how training volume, semantic diversity, and feature representation influence model confidence and decision boundaries when processing real-world user feedback [1], [4].

II. EXPERIMENTAL METHODOLOGY & RESULTS

A. Dataset Construction

The model was trained across three distinct categories:

1. Positive_Praise: 10 distinct examples expressing customer satisfaction.
2. Negative_Complaint: 10 distinct examples detailing product defects or frustration.
3. Urgent_Support: 10 distinct examples highlighting critical operational issues (e.g., locked accounts, system outages).

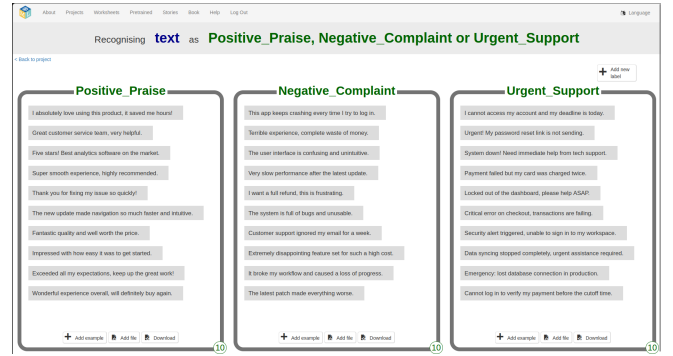


Fig. 1. (*Machine Learning for Kids Training Summary Dashboard showing sample counts per category Positive_Praise: 10, Negative_Complaint: 10, Urgent_Support: 10*)

B. Evaluation Setup & Test Performance

To stress test the model, a test suite of 10 unseen phrases was passed into the inference engine. Table I details the model's predictions, confidence scores, and qualitative analysis.

Test Phrase	True Class	Predicted Class	Confidence	Result	Analysis
This app works wonderfully!	Positive_Praise	Positive_Praise	84%	Correct	Keywords: "wonderfully"
Oh great, another crash right in the middle of my work.	Negative_Complaint	Positive_Praise	99%	Incorrect	Sarcasm error
Can I change my billing address?	Out_of_Vocabulary	Urgent_Support	1%	Incorrect	Guessing blindly
Remarkable stability and smooth performance.	Positive_Praise	Positive_Praise	81%	Correct	Semantic association
Horrible customer service, nobody responded.	Negative_Complaint	Negative_Complaint	84%	Correct	Keywords: "horrible"
Help! I am locked out during an active migration!	Urgent_Support	Urgent_Support	92%	Correct	Keywords: "help!" "locked out"
Thanks a lot for breaking my account right before a presentation.	Negative_Complaint	Positive_Praise	71%	Incorrect	Sarcasm error
Is there a discount for annual subscriptions?	Out_of_Vocabulary	Negative_Complaint	2%	Incorrect	Guessing blindly
System unresponsive, live site is currently failing!	Urgent_Support	Urgent_Support	79%	Correct	Incident patterns: "failing"
Love the clean layout, super user-friendly!	Positive_Praise	Positive_Praise	86%	Correct	Keywords: "Love"

Fig. 2. (*Model Stress-Test Results Across 10 Unseen Test Phrases*)

Category	Correct Predictions	Total Samples	Accuracy (%)
Positive_Praise	3	3	100.00%
Negative_Complaint	1	3	33.33%
Urgent_Support	2	2	100.00%
Out_of_Vocabulary	0	2	0.00%
Overall Performance	6	10	60.00%

Fig. 3. (*Class-Level Accuracy Summary*)

III. DISCUSSION & ANALYSIS

A. Explicit Rules vs. Tagged Learning [1]

If we attempted to construct a text router strictly through explicit programming (using nested IF/ELSE conditions or regular expressions), we would encounter several major hurdles:

- Combinatorial Explosion: Natural language features an extensive array of vocabulary, regional slang, typos, and syntax permutations. Hardcoding rules to catch every combination requires thousands of brittle conditional blocks.
- Context Blindness: Rule-based logic evaluates isolated keywords. For instance, a rule looking for "great" would flag "Oh great, another crash" as positive feedback.
- High Maintenance Costs: Every new product feature or shift in customer language requires software engineers to manually edit and re-test rule bases.

Tagged deep learning avoids these issues by converting text into high-dimensional numerical vectors, allowing the neural network to automatically discover mathematical correlations between word patterns and intent categories.

B. Data Bias & Dataset Quality

When the dataset was limited to 3 examples per class, the model produced unstable predictions with low confidence scores across unseen phrases. Expanding the training set to 10 distinct examples significantly expanded the model's decision boundaries.

Deep learning accuracy relies heavily on volume and representative diversity for two reasons:

- Overcoming Keyword Bias: With sparse data (3 examples), a model assigns disproportionate statistical weight to individual surface words (e.g., assuming "great" always implies praise).
- Handling Out-of-Vocabulary Contexts: As demonstrated in Test Cases #3 and #8, when an input contains zero semantic overlap with the training set, the neural network cannot map the vector correctly. While it still outputs a prediction, the near-zero confidence scores (1% and 2%) mathematically indicate that the input falls into an unrepresented domain.

IV. ARCHITECTURAL PIPELINE ANALYSIS (FEATURE EXTRACTION)

In the Machine Learning for Kids natural language processing workflow, feature extraction occurs immediately after raw text ingestion and before the extracted vectors are evaluated by the dense neural network layers. Rather than relying on manually engineered rules or static dictionary lookups, the model processes text through a multi-stage sequential pipeline:

1. Raw Text Ingestion & Preprocessing: The pipeline accepts raw string inputs (such as user feedback messages). Standard text cleaning procedures including lowercasing, whitespace normalization, and special character stripping are applied to ensure input standardization.
2. Tokenization & Numerical Mapping: The cleaned string is broken down into constituent sub-units or words (tokens). Because mathematical neural networks cannot operate directly on natural language strings, each unique token is mapped to a corresponding numerical identifier within a pre-established vocabulary index.
3. Feature Extraction & Embedding Space: This stage represents the primary feature extraction boundary. The sequence of numerical tokens is projected into a high-dimensional continuous vector space (word embeddings) [4]. This process automatically extracts rich semantic features, encoding spatial distances, syntactic relationships, and co-occurrence frequencies across words without requiring human feature engineering.
4. Neural Network Classification: The extracted feature vectors are passed through the hidden layers of the feedforward neural network. Within these layers, weighted connections and non-linear activation functions evaluate the feature representations against patterns learned during the training phase.
5. Probability Output (Inference): Finally, the activation values of the output layer are passed through a Softmax function [4]. This converts raw logit scores into a normalized probability distribution, designating the category with the highest confidence score as the predicted class label (e.g., classifying "The app works wonderfully!" into Positive_Praise with 84% confidence).

V. CONCLUSION

This exercise highlights both the strengths and vulnerabilities of shallow neural network classifiers. While supervised neural networks easily outperform traditional rule-based scripts on standard inputs, their accuracy heavily relies on the quality, volume, and context of their training data. To mitigate real-world failures caused by sarcasm and out-of-vocabulary queries, practical implementations must integrate broader datasets, fallback confidence thresholds, and intent classes for administrative queries.

References

- [1] D. Lane, *Machine Learning for Kids: A Project-Based Introduction to Artificial Intelligence*, San Francisco, CA, USA: No Starch Press, 2020.
- [2] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.
- [3] Machine Learning for Kids, "Recognising Text - Project Documentation and Architecture," IBM Developer / Dale Lane, 2024. [Online].
- [4] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Dive into Deep Learning*, Cambridge University Press, 2023.